

Semana 11 - Sesión 1 (Sesión 21)

Introducción a pandas y manejo de datos

Unidad V: Clases y análisis de datos

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 11, Sesión 1 (Sesión 21)

Contexto

Fundamentos de pandas

Lectura e inspección

Selección y filtrado

Datos faltantes

Agrupación y resumen

Visualización y exportación

Ejercicio integrador

Cierre

Contexto

Borrón y cuenta nueva: Unidad V

- El syllabus ubica después de la Semana 10 una etapa de consolidación y luego el inicio de la **Unidad V: Clases y análisis de datos**.
- Por el ajuste de calendario, esta sesión retoma el contenido previsto para el bloque siguiente: **manejo de datos en Python con pandas**.
- La idea es volver a revisar pandas desde la base, sin asumir dominio previo.
- Esta P1 será una introducción ordenada; la P2 puede quedar como laboratorio aplicado.

Meta de la sesión

Leer, inspeccionar, limpiar, filtrar y resumir datos tabulares usando pandas.

- **Semana 7:** apareció pandas como una introducción opcional para leer datos tabulares.
- **Semana 8:** se trabajaron fundamentos: **Series**, **DataFrame**, selección, filtrado, **groupby** y datos faltantes.
- **Semana 9:** se usó pandas como apoyo breve junto con Matplotlib avanzado.
- **Ahora:** reorganizamos esos contenidos como una clase inicial coherente de análisis de datos.

Al terminar la sesión, deberían poder:

- importar pandas con el alias estándar `pd`;
- crear una **Series** y un **DataFrame**;
- leer un archivo CSV con `pd.read_csv`;
- inspeccionar una tabla con `head`, `shape`, `columns`, `info` y `describe`;
- seleccionar columnas y filas;
- filtrar datos usando condiciones;
- detectar y tratar valores faltantes;
- resumir datos usando `groupby`;
- hacer visualizaciones rápidas y exportar resultados.

Fundamentos de pandas

- En física y astronomía muchas mediciones llegan como **tablas**: columnas, unidades, categorías y valores faltantes.
- NumPy es excelente para arreglos numéricos homogéneos.
- pandas agrega una capa tabular con nombres de columnas, índices y herramientas de análisis exploratorio.

Idea central

Un **DataFrame** es una tabla con columnas etiquetadas. Cada columna puede representar una variable física, observacional o experimental.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
```

- `pd` es el alias estándar de pandas.
- `np` se mantiene para cálculos numéricos.
- `plt` se usa para visualizar resultados.

Fundamentos de pandas

Series: una columna con índice

```
1 import pandas as pd
2
3 s = pd.Series(
4     [1.5, 2.0, 0.8],
5     index=["objeto1", "objeto2", "objeto3"]
6 )
7
8 s
```

# 0	
objeto1	1.5
objeto2	2.0
objeto3	0.8

Lectura: una **Series** funciona como un vector, pero cada elemento puede tener una etiqueta.

Fundamentos de pandas

DataFrame: tabla bidimensional

```
1 df = pd.DataFrame({
2     "masa": [1.5, 2.0, 0.8],
3     "velocidad": [10.5, 8.2, 15.3]
4 })
5
6 df
```

	# masa	# velocidad
0	1.5	10.5
1	2.0	8.2
2	0.8	15.3

- Cada columna tiene nombre.
- Cada fila representa una observación u objeto.
- pandas permite mezclar texto y números en una misma tabla.

Fundamentos de pandas

Crear columnas derivadas

```
1 # Energía cinética:
2 #  $E_c = 1/2 m v^2$ 
3 df["energia_cinetica"] = (
4     0.5 * df["masa"] *
5     df["velocidad"]**2
6 )
7
8 df
```

	# masa	# velocidad	# energia_cinetica
1	1.5	10.5	82.6875
2	2.0	8.2	67.24
Fila 3	0.8	15.3	93.63600000000002

Punto clave: las operaciones sobre columnas se aplican de forma vectorizada, parecido a NumPy.

Fundamentos de pandas

Crear categorías a partir de cálculos

```
1 df["categoria"] = pd.cut(  
2     df["energia_cinetica"],  
3     bins=[0, 70, 100],  
4     labels=["Baja", "Alta"]  
5 )  
6  
7 df
```

	# masa	# velocidad	# energia_cinetica	📦 categoria
1		1.5	Missing value	Alta
2		2.0	8.2	Baja
Fila 3		0.8	15.3	Alta

Lectura e inspección

```
1 url = (  
2     "https://raw.githubusercontent.com/"  
3     "mwaskom/seaborn-data/master/planets.csv"  
4 )  
5  
6 planets = pd.read_csv(url)  
7  
8 print(planets.head())
```

- `pd.read_csv` carga datos desde archivo local o URL.
- `head()` muestra las primeras filas para revisar estructura.

Lectura e inspección

Inspección inicial

```
1 print(planets.shape)
2 print(planets.columns)
3
4 planets.info()
```

Preguntas que responde

- ¿Cuántas filas y columnas hay?
- ¿Cómo se llaman las columnas?
- ¿Qué tipos de datos aparecen?
- ¿Hay columnas con valores faltantes?

Lectura e inspección

Resumen estadístico

```
1 df.describe()
2
3 df["energia_cinetica"].mean()
```

	# masa	# velocidad	# energia_cinetica
count	3.0	3.0	3.0
mean	1.4333333333333333	11.333333333333334	81.18783333333334
std	0.6027713773341707	3.6226141573915016	13.261747776342819
min	0.8	8.2	67.24
25%	1.15	9.35	74.96375
50%	1.5	10.5	82.6875
75%	1.75	12.9	88.16175000000001
max	2.0	15.3	93.63600000000002

- `describe()` entrega conteo, media, desviación estándar, mínimos y cuartiles.
- Es el primer diagnóstico numérico de una tabla.

Selección y filtrado

Selección y filtrado

Seleccionar columnas

```
1 # Una columna
2 df["masa"]
3
4 # Varias columnas
5 subset = df[["masa", "velocidad"]]
6
7 subset
```

	# masa	# velocidad
0	1.1622909903858079	6.294779128285901
1	1.2575392838565098	5.195686868016088
2	1.5875412989737128	12.157919722098546
3	1.2514011420766056	13.672540022650423
4	1.1538870112094566	16.95447565316892
5	1.4731483172445072	13.015316641086615
6	0.9072892094622428	12.942605710820246
7	0.74477645177605	18.15180828289533
8	1.4300533214655435	13.98126996753548
9	0.686840808473431	6.4791834216107

Regla práctica: una columna se selecciona con un string; varias columnas con una lista de strings.

```
1 # Por posición
2 print(planets.iloc[0])
3 print(planets.iloc[0:5])
4
5 # Por condición con loc
6 transito = planets.loc[planets["method"] == "Transit"]
7
8 print(transito.head())
```

- **iloc**: usa posiciones numéricas.
- **loc**: usa etiquetas o condiciones lógicas.

Selección y filtrado

Filtros simples

```
1 # Filtrado con una condición
2 altas_temp = df[
3     df["temperatura"] > 300
4 ]
5
6 altas_temp
```

#	velocidad	#	temperatura	📄	categoria
	13.015316641086615		330		Liquido
	7.80473808300243		368		Liquido
	16.036247391526743		363		Liquido
	14.312656446831161		382		Gaseoso
	14.2988753285948		389		Gaseoso
	19.257432372696933		306		Liquido
	15.414132414861374		345		Liquido
	5.531101211711803		362		Liquido
	10.711592605859591		386		Gaseoso
	16.888192119760735		363		Liquido

Idea: un filtro crea una máscara **True/False**; pandas conserva solo las filas verdaderas.

Selección y filtrado

Filtros múltiples

```
1 mask = (  
2     (planets["method"] == "Radial Velocity") &  
3     (planets["mass"] > 1) &  
4     (planets["distance"] < 100)  
5 )  
6  
7 seleccion = planets.loc[  
8     mask,  
9     ["method", "orbital_period", "mass", "distance"]  
10 ]  
11  
12 print(seleccion.head())
```

- Use & para condiciones simultáneas.
- Cada condición debe ir entre paréntesis.

Datos faltantes

Datos faltantes

Detectar valores faltantes

```
1 import numpy as np
2
3 df.loc[[1, 3], "temperatura"] = np.nan
4
5 df.isnull().sum()
```

# 0	
masa	0
velocidad	0
temperatura	50
categoria	0

Por qué importa: muchos datasets reales tienen mediciones incompletas; primero se identifica qué falta.

Datos faltantes

Eliminar filas incompletas

```
1 df_dropped = df.dropna()  
2  
3 df_dropped
```

	# masa	# velocidad	# temperatura	categoria
0	0.702945440225168	6.3865191775676955	109.0	Solido
1	0.9690671393777127	6.048542373822548	23.0	Solido
2	1.333310990489152	15.240299023778565	234.0	Solido
3	1.9231138793475124	19.90359211556003	353.0	Liquido
6	1.105603194347084	11.443547055237433	220.0	Solido
7	1.95634581987672	10.806381878743224	352.0	Liquido
10	1.3044770518799047	11.894737405806785	390.0	Gaseoso
14	1.3147638161813286	17.159520563824376	311.0	Liquido
17	0.6504724438774485	18.12720739180797	340.0	Liquido
18	1.1617941760020265	8.821518400931424	39.0	Solido

50 rows x 4 cols 10 per page < < Page 1 of 5 > > 🔍 📄 🗨️ ⋮

- `dropna()` elimina filas con valores faltantes.
- Se debe evaluar si conviene perder filas o usar otra estrategia.

Datos faltantes

Rellenar valores faltantes

```
1 numeric_means = df.select_dtypes(  
2     include=[np.number]  
3 ).mean()  
4  
5 df_filled = df.fillna(numeric_means)  
6  
7 df_filled
```

	# masa	# velocidad	# temperatura	📦 categoria
0	0.702945440225168	6.3865191775676955	109.0	Solido
1	0.9690671393777127	6.048542373822548	230.5	Solido
2	1.333310990489152	15.240299023778565	234.0	Solido
3	1.9231138793475124	19.90359211556003	230.5	Liquido
4	0.8943455857442097	12.41997460894105	230.5	Solido

Cuidado: rellenar con la media puede ser útil para práctica, pero no siempre es físicamente correcto.

Agrupación y resumen

Agrupación y resumen

Agrupar con **groupby**

```
1 resumen = df.groupby("categoria")  
2     "temperatura"  
3 ].mean()  
4  
5 resumen
```

categoria	# temperatura
Solido	127.55737704918033
Liquido	330.65625
Gaseoso	389.0

Interpretación: `groupby` separa la tabla por categorías y aplica una operación a cada grupo.

Agrupación y resumen

Agrupar con varias estadísticas

```
1 resumen = df.groupby("categoria").agg({  
2     "masa": ["mean", "min", "max"],  
3     "velocidad": ["mean", "std"],  
4     "temperatura": "mean"  
5 })  
6  
7 resumen
```

- **count**: cantidad de datos no nulos.
- **mean**: promedio.
- **median**: mediana.
- **max**: valor máximo.

Agrupación y resumen

Ordenar resultados

```
1 conteos = df["categoria"].value_counts()
2 print(conteos.head())
3
4 categorias_comunes = conteos.sort_values(ascending=False)
5 print(categorias_comunes.head())
```

Uso típico

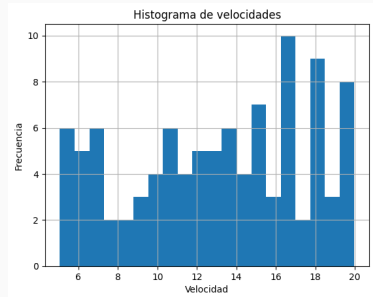
Ordenar ayuda a identificar categorías dominantes, valores extremos o métodos más frecuentes.

Visualización y exportación

Visualización y exportación

Histograma rápido

```
1 df["velocidad"].hist(bins=20)
2
3 plt.title("Histograma de velocidades")
4 plt.xlabel("Velocidad")
5 plt.ylabel("Frecuencia")
6 plt.show()
```

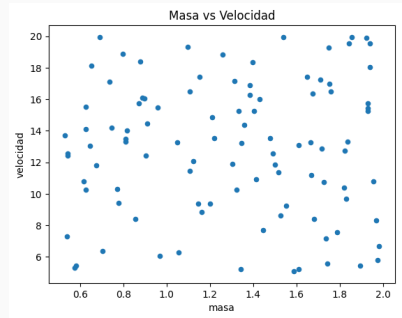


- pandas se integra directamente con Matplotlib.
- Para gráficos más controlados, se puede usar `plt` explícitamente.

Visualización y exportación

Scatter con Matplotlib

```
1 df.plot(  
2     x="masa",  
3     y="velocidad",  
4     kind="scatter",  
5     title="Masa vs Velocidad"  
6 )  
7  
8 plt.show()
```



Visualización y exportación

Exportar resultados

```
1 df_filled.to_csv("datos_limpios.csv", index=False)
2 resumen.to_csv("resumen_por_categoria.csv")
```

Para qué sirve

Después de limpiar, filtrar o resumir datos, conviene guardar una versión reproducible del resultado.

Ejercicio integrador

Use el dataset `planets.csv` y realice:

1. Leer el CSV con `pd.read_csv`.
2. Mostrar `head`, `shape`, `columns` e `info`.
3. Crear un DataFrame limpio usando columnas relevantes.
4. Filtrar planetas detectados por velocidad radial.
5. Calcular distancia promedio y masa promedio por método.
6. Graficar masa versus periodo orbital.
7. Exportar el subconjunto filtrado.

Ejercicio integrador

Solución guiada: lectura, limpieza y filtro

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 url = (
5     "https://raw.githubusercontent.com/"
6     "mwaskom/seaborn-data/master/planets.csv"
7 )
8
9 planets = pd.read_csv(url)
10
11 print(planets.head())
12 print(planets.shape)
13 print(planets.columns)
14 planets.info()
15
16 cols = ["method", "orbital_period", "mass", "distance", "year"]
17 datos = planets[cols].dropna()
18
19 rv = datos[datos["method"] == "Radial Velocity"]
20 print(rv.head())
```

Ejercicio integrador

Solución guiada: resumen, gráfico y exportación

```
1 resumen = datos.groupby("method").agg({
2     "distance": "mean",
3     "mass": "mean",
4     "orbital_period": "median"
5 })
6
7 print(resumen)
8
9 plt.figure(figsize=(7, 5))
10 plt.scatter(rv["orbital_period"], rv["mass"], alpha=0.6)
11 plt.xlabel("Periodo orbital [días]")
12 plt.ylabel("Masa [M_J]")
13 plt.title("Planetas detectados por velocidad radial")
14 plt.grid(True, alpha=0.3)
15 plt.show()
16
17 rv.to_csv("planetas_velocidad_radial.csv", index=False)
```

Cierre

- Olvidar importar pandas como `pd`.
- Usar `df[column]` sin comillas: debe ser `df["column"]`.
- Combinar condiciones sin paréntesis.
- Usar `and` en vez de `&` para filtros de pandas.
- Calcular promedios sin revisar valores faltantes.
- Modificar una tabla sin guardar una copia cuando se quiere conservar el original.

- pandas permite trabajar con datos tabulares de forma eficiente.
- **DataFrame** es la estructura central.
- El flujo básico es:
leer → inspeccionar → limpiar → filtrar → resumir → visualizar
- La próxima sesión puede profundizar en ejercicios aplicados y análisis estadístico simple.

Practicar:

- leer CSV locales y remotos;
- revisar `head`, `info`, `describe`;
- seleccionar columnas;
- filtrar filas;
- limpiar datos faltantes;
- usar `groupby`;
- hacer un histograma y un scatter;
- guardar resultados con `to_csv`.